

Fuja para o Castelo

Jogo educativo infantil embarcado
Documentação inicial - Semana 1

Laboratório de Processadores

Kit Freenove FNK0054 e Raspberry Pi

Integrantes

Claudio Lucio Cunha da Silva

Lays Vieira Zandomingos

Solano Omar Oliveira do Nascimento

Resumo executivo

O projeto propõe um jogo educativo para crianças de 7 a 10 anos. O jogador percorre uma fase até um castelo e encontra portais com perguntas. Respostas corretas geram moedas, escudo, vidas e outras vantagens. O sistema será executado em Raspberry Pi e explorará diversos componentes do kit Freenove FNK0054, combinando processamento gráfico, aquisição de entradas, controle de atuadores e medição de desempenho.

1. Motivação

Jogos educativos podem tornar o aprendizado mais interativo ao associar conteúdos escolares a desafios, recompensas e respostas imediatas. O projeto Fuja para o Castelo propõe um jogo direcionado inicialmente a crianças de 7 a 10 anos, no qual o jogador controla um personagem, supera obstáculos e responde perguntas de matemática, português, ciências e lógica.

As respostas corretas concedem benefícios dentro do jogo, como moedas, escudo, vidas extras e vantagens temporárias. Em caso de erro, o sistema apresenta uma explicação curta e evita punições excessivas, preservando o caráter educativo da experiência.

O jogo será executado em um Raspberry Pi e integrado aos componentes do kit Freenove FNK0054. Botões, joystick, displays, LEDs, buzzer, servo motor e sensores serão utilizados para criar uma interação física e multimodal. Dessa forma, o projeto explora simultaneamente processamento gráfico, lógica de jogo, leitura de entradas, controle de atuadores, temporização e coleta de métricas.

Além da finalidade educativa, o projeto permitirá avaliar o comportamento do sistema quando diferentes periféricos operarem ao mesmo tempo. Serão observados taxa de quadros, uso de CPU e memória, latência das entradas, tempo de resposta das perguntas e estabilidade dos atuadores.

2. Objetivos e escopo

2.1 Objetivo geral

Desenvolver um jogo educativo infantil executado em uma plataforma embarcada, integrando perguntas, recompensas e componentes do kit Freenove FNK0054 para oferecer uma experiência interativa, física e adequada ao aprendizado.

2.2 Objetivos específicos

- Criar uma fase jogável com linguagem visual simples e adequada ao público infantil.
- Apresentar perguntas educacionais durante a progressão do jogador.
- Recompensar respostas corretas com benefícios dentro do jogo.
- Apresentar explicações simples após respostas incorretas.
- Permitir respostas por botões físicos e movimentação por joystick.
- Utilizar LEDs, display, buzzer e servo para fornecer retorno imediato.
- Manter o jogo responsivo durante a operação simultânea dos periféricos.
- Registrar acertos, erros, tempo de resposta, FPS, CPU e memória.
- Permitir a inclusão de novas perguntas sem alteração do código principal.

2.3 Público-alvo

Crianças de 7 a 10 anos, com perguntas de dificuldade inicial e intermediária. A faixa poderá ser refinada após testes com usuários e avaliação pedagógica.

2.4 Escopo da primeira versão

A primeira versão deverá conter uma fase, personagem, obstáculos, três ou mais portais educativos, perguntas de múltipla escolha, recompensas, condição de vitória, condição de derrota e suporte a teclado. A integração física será implementada incrementalmente sobre a interface de hardware definida no protótipo.

3. Especificação de requisitos

3.1 Requisitos funcionais

ID	Requisito
RF01	O sistema deve executar o jogo no Raspberry Pi.
RF02	O jogador deve movimentar o personagem e realizar saltos.
RF03	O jogo deve apresentar obstáculos e detectar colisões.
RF04	O jogo deve apresentar perguntas durante o percurso.
RF05	Cada pergunta deve possuir de duas a quatro alternativas.
RF06	O jogador deve responder usando teclado ou botões físicos.
RF07	O sistema deve validar a resposta selecionada.
RF08	O sistema deve fornecer retorno visual e sonoro para acertos e erros.
RF09	Respostas corretas devem conceder moedas ou vantagens.
RF10	Respostas incorretas devem apresentar uma explicação curta.
RF11	O sistema deve contabilizar acertos, erros e sequência de acertos.
RF12	O sistema deve medir o tempo de resposta de cada pergunta.
RF13	O sistema deve evitar repetição imediata da mesma pergunta.
RF14	As perguntas devem ser carregadas de arquivo externo.
RF15	O sistema deve apresentar um resumo ao final da partida.
RF16	Um servo deve representar fisicamente o progresso na fase.
RF17	LEDs devem indicar acerto, erro, dano e vitória.
RF18	Um buzzer deve produzir sons distintos para eventos relevantes.
RF19	Um display externo deve apresentar pontuação ou informações da pergunta.
RF20	O sistema deve possuir modo de diagnóstico de periféricos.

3.2 Requisitos não funcionais

ID	Categoria	Requisito
RNF01	Desempenho	O jogo deve manter média

		mínima de 30 FPS em operação normal.
RNF02	Latência	Entradas digitais devem ser reconhecidas em até 100 ms.
RNF03	Aquisição	Botões devem ser amostrados com frequência mínima de 50 Hz.
RNF04	Confiabilidade	Os botões devem possuir tratamento de debounce.
RNF05	Concorrência	Leituras de sensores não devem bloquear a renderização.
RNF06	Modularidade	A lógica do jogo deve ser independente dos drivers físicos.
RNF07	Portabilidade	Deve existir modo teclado sem hardware conectado.
RNF08	Observabilidade	FPS, CPU, memória e eventos devem ser registrados.
RNF09	Robustez	A falha de um periférico secundário não deve encerrar o jogo.
RNF10	Segurança	A pinagem deve respeitar os limites elétricos do Raspberry Pi.
RNF11	Manutenção	Pinagem, montagem e dependências devem ser documentadas.
RNF12	Encerramento	Atuadores devem retornar a estado seguro ao encerrar.
RNF13	Usabilidade	Textos e comandos devem ser legíveis e adequados ao público infantil.
RNF14	Extensibilidade	Novas perguntas devem poder ser adicionadas via JSON.

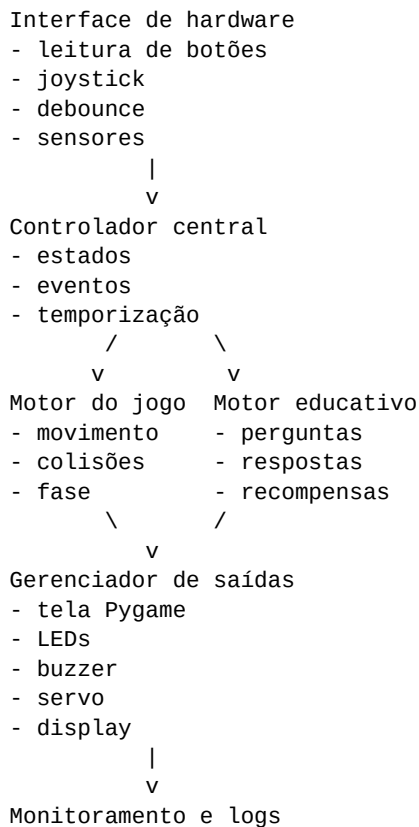
3.3 Regras de recompensa

Evento	Recompensa inicial
Resposta correta	10 moedas
Resposta em até 5 segundos	5 moedas adicionais
Três acertos consecutivos	Escudo contra uma colisão
Cinco acertos consecutivos	Uma vida extra
Resposta incorreta	Explicação e nenhuma perda de vida

4. Arquitetura proposta

A arquitetura separa o jogo, o conteúdo educativo, o acesso ao hardware e o monitoramento. Essa separação permite executar o protótipo em um computador comum e, posteriormente, substituir o controle por teclado pelos dispositivos físicos sem modificar as regras principais.

Entradas físicas ou teclado
|
v



4.1 Módulos

- ``src/main.py``: laço principal e integração dos módulos.
- ``src/game/``: entidades, física, colisões e estado da partida.
- ``src/education/``: carregamento e seleção de perguntas.
- ``src/hardware/``: interface abstrata e implementação por teclado.
- ``src/monitoring/``: coleta de FPS, CPU e memória.
- ``data/questions.json``: banco externo de perguntas.

4.2 Estados principais

- ``PLAYING``: personagem percorre a fase.
- ``QUESTION``: jogo apresenta pergunta e aguarda resposta.
- ``FEEDBACK``: acerto ou erro é apresentado por tempo limitado.
- ``GAME_OVER``: vidas esgotadas.
- ``VICTORY``: personagem alcança o castelo.

5. Plano de integração do hardware

5.1 Componentes previstos

Componente	Aplicação
Joystick	Movimentação do personagem e menus
Quatro botões	Alternativas A, B, C e D
LED RGB	Verde para acerto, vermelho para erro e azul para bônus
Buzzer passivo	Efeitos de acerto, erro, dano e vitória
Servo motor	Indicador físico do progresso até o castelo

LCD	Pontuação, vidas, pergunta resumida ou estado
Potenciômetro	Ajuste de dificuldade ou volume
Fotoresistor	Alteração do cenário entre dia e noite
Sensor ultrassônico	Comando especial por aproximação
Matriz ou display	Vidas, moedas ou sequência de acertos

5.2 Integração incremental

1. Validar o protótipo completo por teclado.
2. Substituir as alternativas 1 a 4 por quatro botões com debounce.
3. Integrar LED RGB e buzzer aos eventos de resposta.
4. Integrar servo ao progresso percentual da fase.
5. Integrar LCD à pontuação e ao estado atual.
6. Integrar joystick à movimentação.
7. Adicionar sensores analógicos e ambientais.
8. Executar todos os dispositivos simultaneamente e medir desempenho.

5.3 Estratégia de concorrência

O laço principal manterá a renderização e a lógica do jogo. Sensores de leitura lenta poderão operar em threads separadas, enviando eventos por uma fila segura. Atuadores serão atualizados apenas quando o estado mudar, evitando operações desnecessárias a cada quadro.

6. Plano de testes

6.1 Testes funcionais

- Verificar movimentação, salto e limites da tela.
- Verificar colisão com obstáculos.
- Verificar abertura de perguntas nos portais.
- Verificar seleção das quatro alternativas.
- Verificar moedas, escudo, vidas e sequências.
- Verificar vitória, derrota e reinício.
- Verificar carregamento e validação do JSON.

6.2 Testes de hardware

Cada componente deverá possuir um programa isolado de diagnóstico antes da integração. Os testes devem confirmar pinagem, sentido lógico, debounce, tempo de resposta e estado seguro de encerramento.

6.3 Testes de desempenho

Cenário	Configuração
Referência	Apenas jogo e teclado
Carga baixa	Botões e LEDs
Carga média	Botões, LEDs, buzzer, servo e LCD
Carga alta	Todos os periféricos e sensores
Sobrecarga	Todos os periféricos, logs detalhados e carga gráfica ampliada

Métricas: FPS médio e mínimo, CPU, memória, latência média e máxima, tempo de leitura dos sensores, eventos perdidos e variação temporal dos atuadores.

7. Cronograma inicial

Semana	Entrega
1	Repositório, documentação, Release v0.1.0 e protótipo inicial
2	Refinamento da fase, banco de perguntas e sistema de recompensas
3	Integração de botões, LED RGB e buzzer
4	Integração de joystick, servo e display
5	Integração dos sensores adicionais
6	Concorrência, logs e monitoramento de desempenho
7	Testes com carga, correções e melhoria da experiência infantil
8	Relatório final, demonstração e Release estável

8. Protótipo inicial entregue

O pacote desta entrega contém um protótipo funcional em Pygame com movimentação, salto, obstáculos, portais educativos, perguntas em JSON, recompensas, vitória, derrota, coleta de métricas e uma camada abstrata para a integração futura do hardware.

Arquivo	Responsabilidade
src/main.py	Laço principal, estados, renderização e regras do protótipo.
src/education/question_bank.py	Leitura e seleção das perguntas.
src/hardware/interface.py	Contrato comum para entradas e saídas físicas.
src/hardware/keyboard.py	Modo de desenvolvimento por teclado.
src/hardware/freenove_stub.py	Ponto de extensão para o kit FNK0054.
src/monitoring/performance.py	Registro de FPS, CPU e memória.
data/questions.json	Banco inicial de perguntas.

9. Entrega e versionamento

A versão sugerida para a primeira Release é v0.1.0, com o título “Entrega da Semana 1 - Documentação e protótipo inicial”. A Release deve incluir este PDF e apontar para o commit que contém os arquivos Markdown e o código do protótipo.